

Evolutionary Weight Optimization for Variational Quantum Classifiers

1st Matheus Hopper Jansen Costa

Centro de Informática

Universidade Federal de Pernambuco (UFPE)

Recife, Brazil

mhjc@cin.ufpe.br

2nd Fernando M de Paula Neto

Centro de Informática

Universidade Federal de Pernambuco (UFPE)

Recife, Brazil

fernando@cin.ufpe.br

Abstract—Variational Quantum Circuits (VQCs) are viable alternatives for implementing quantum circuits in current quantum processors. VQCs are reduced circuits with parameters that are trained to minimize a loss function for a given problem. In this work, we propose an approach to optimize the parameters of a VQC using Genetic Algorithms (GAs). We explored different GA configurations, on 3 databases - Sonar, Cancer and Banknotes datasets, comparing the proposed approach with 2 classical optimizers based on gradient descent. The results show that genetic algorithms can effectively optimize the weights of variational circuits to achieve high classification accuracy with convergence speed up to 7.3 times (in Sonar Database), 4.8 times (Cancer Database), and 3.7 times (Banknote Database) compared with the traditional optimizers. The experimental results indicate that the genetic algorithm achieves performance metrics that are significantly superior or at least comparable to those obtained by gradient descent-based optimizers, demonstrating the potential and robustness of evolutionary strategies in training variational quantum circuits.

Index Terms—Variational quantum circuits, Genetic algorithms, Quantum Circuit Parameters Optimization.

I. INTRODUCTION

Current quantum processors are too noisy and have a low scale, with few available computing resources. These devices are called NISQ (noisy intermediate-scale quantum) devices. Therefore, quantum algorithms that have many quantum gates and require many qubits (the quantum bits, the minimum quantum processing units) cannot run on these devices, as they generate a very pronounced noise at their outputs. Achieving noise-tolerant quantum computation is essential for the reliable execution of intricate algorithms, needing encoding information across a substantial array of redundant qubits. Such redundancy enables the deployment of quantum error correction codes, a potent strategy for mitigating errors inherent in contemporary quantum hardware [1]. Insufficient qubit count on NISQ devices renders effective error correction impractical [2]. Consequently, successful execution on NISQ devices is dependent on optimizing factors such as circuit depth, width, and the total number of operations [3].

One way to get around the use of very large algorithms is to create small quantum circuits that are adapted to solve a certain task by adjusting their internal parameters. Within this

perspective, variational quantum circuits (VQC) emerge [4]. VQCs are composed of parametric quantum gates that govern the algorithm behavior and exploit the principles of quantum mechanics, particularly superposition and entanglement. These phenomena enable VQCs to explore a vast solution space probabilistically, offering a potential advantage over classical algorithms in learning from complex datasets [5].

By harnessing the principles of superposition and entanglement, VQCs can potentially overcome the inherent limitations of classical neural networks, particularly in terms of scalability and computational efficiency for specific problem domains. This ability comes from the inherent parallelism within quantum computations, allowing VQC architectures to explore a significantly larger solution space compared to their classical counterparts [6].

Beyond hardware constraints, training models that have parameters distributed between different quantum gates that compute completely different functions leads to the challenge of finding efficient ways to find optimal results. Optimizing the weights of VQCs presents a significant challenge due to the high-dimensional search space. Here, in this paper, we propose an approach that leverages the use of genetic algorithms (GAs), an evolutionary technique, to optimize VQC weights. This method offers a compelling alternative to traditional optimizers. We demonstrate the effectiveness of our approach by tackling classification problems on three diverse datasets from the UCI Machine Learning Repository: sonar [7], banknotes [8] and breast cancer [9]. In addition, we explore the design of six distinct VQC architectures, investigating how these architectures influence weight updates within the network.

The subsequent sections of this work are organized as follows: Section 2 reviews related works on applying GAs to optimize neural networks. Section 3 delves into the theoretical framework of VQCs. Section 4 details the principles of GAs. Section 5 presents our proposed GA-based VQC optimization algorithm. Section 6 provides comprehensive results obtained on the aforementioned datasets. Finally, Section 7 draws conclusions and discusses future work directions.

II. RELATED WORKS

Articles proposing parameter optimization using genetic algorithms are quite old and have consolidated results [10],

[11], [12]. Generally, their results point to faster convergence time and better training.

Regarding the training of variational quantum circuit parameters, there are few works that make use of gradient-free methods. One of them was proposed in [13], but is applied only to the Iris base and only compares its results with an optimizer or does not compare with any optimizer [14]. In [15], the genetic algorithm is used as an approach to optimize a quantum algorithm of the type quantum approximate optimization algorithm. Other related works use genetic and other evolutionary algorithms to optimize variational quantum circuit architectures, not their weights [16], [17]. Their focus was on finding optimal circuit structures, such as the number and arrangement of quantum gates, rather than tuning specific parameters for a given task.

In contrast, our approach optimizes parameters (weights) of different quantum circuits, allowing each circuit to achieve better predictive accuracy by fine-tuning parameters for specific datasets. Optimizing only the architecture may result in good structural potential, but it lacks the necessary parameter adjustments to fully exploit that potential for specific tasks.

The need for efficient parameter optimization of VQCs aligns well with the objectives of genetic algorithms and other evolutionary strategies, as these approaches do not rely on gradient calculations, which are often challenging in quantum environments due to issues like barren plateaus. Genetic algorithms can help navigate complex, high-dimensional parameter spaces more effectively, especially for quantum systems where traditional gradient-based approaches struggle.

This connection between optimization methods and quantum systems also lays the foundation for the development of quantum neurons. The Quantum Neuron concept is built on leveraging quantum computation's intrinsic properties, such as superposition and entanglement, which could enhance the representational power of neural networks. The training of such neurons, however, faces similar optimization challenges as variational quantum circuits, making evolutionary algorithms an attractive option. [18]

Recently a library for training VQCs with GAs was release in [19], with no execution yet applied to databases.

III. QUANTUM NEURON

A. Quantum computation

The qubit is a two-dimensional vector in the complex vector space $\mathbb{C}^2$. It can be in superposition of states, *ie.* in position 0 or position 1 at the same time, if one considers the canonical basis as 0 and 1. Any qubit $|g\rangle^1$ can be written as a linear combination of vectors (or states) of $\mathbb{C}^2$ computational basis $|0\rangle = [1, 0]^T$ and $|1\rangle = [0, 1]^T$ as viewed in Equation 1.

$$|g\rangle = \alpha |0\rangle + \beta |1\rangle \quad (1)$$

A string of qubits is represented mathematically by the tensor operator, $\otimes$. The tensor operator is used to represent quantum systems with two or more qubits $|g\rangle = |ab\rangle = |a\rangle \otimes |b\rangle$.

¹The notation $|\cdot\rangle$ for quantum states vectors are proposed by Dirac and it is commonly used to represent complex-valued vectors.

The quantum states are modified by quantum operators which change the amplitude values of the qubits. A quantum operator U over n qubits is a unitary complex matrix of order $2^n \times 2^n$.

Any single-qubit quantum gate can be described as a combination of gates R_x , R_y , and R_z , described respectively in Equations 2, 3 and 4, or even in the form described by $U(\theta, \beta, \gamma)$ in Equation 5.

$$R_x(\theta) = \begin{pmatrix} \cos(\theta/2) & -i \cdot \sin(\theta/2) \\ -i \cdot \sin(\theta/2) & \cos(\theta/2) \end{pmatrix} \quad (2)$$

$$R_y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix} \quad (3)$$

$$R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix} \quad (4)$$

$$U(\theta, \beta, \gamma) = \begin{pmatrix} \cos(\theta/2) & -e^{i\gamma} \cdot \sin(\theta/2) \\ e^{i\beta} \sin(\theta/2) & e^{i(\beta+\gamma)} \cos(\theta/2) \end{pmatrix} \quad (5)$$

The **CNOT** gate is a two-qubit operator consisting of a control qubit and a target qubit. Its function is to apply the **X** operator to the target qubit based on the value of the control qubit. When the control qubit is set to 1, the $X = R_x(\theta = \pi)$ operator is applied to the target qubit. The matrix representation for the computational basis is provided in Equation 6.

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad \begin{array}{l} \text{CNOT} |00\rangle = |00\rangle \\ \text{CNOT} |01\rangle = |01\rangle \\ \text{CNOT} |10\rangle = |11\rangle \\ \text{CNOT} |11\rangle = |10\rangle \end{array} \quad (6)$$

Quantum operations can be depicted using quantum circuits. In this graphical representation, qubits are shown as wires, and quantum operators are represented by boxes. Similarly to classical circuits, the execution flow proceeds from left to right.

Figure 1 illustrates a quantum circuit example that includes a **CNOT** gate, with the control qubit represented by a filled circle, and an **X** operator. In addition, the circuit has a controlled R_y gate and a R_x gate.

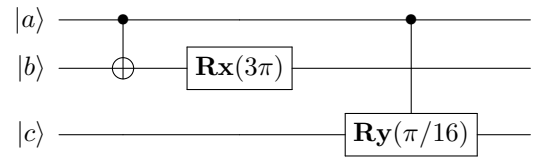


Fig. 1. An example of quantum circuit with three qubits, $|a\rangle$, $|b\rangle$, $|c\rangle$, one **CNOT** operator and two rotation gates, R_y and R_x , one of them being controlled (controlled rotation).

Measurement is an irreversible process that results in the partial or complete loss of information about quantum states. For a qubit in the state $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$, measurement collapses the state to either $|0\rangle$ with a probability of $|\alpha|^2$ or to $|1\rangle$ with a probability of $|\beta|^2$. For a composite and superposed quantum state $|g\rangle$, the probability of observing the state $|i\rangle$ is given by $|\langle g|i\rangle|^2$. More details on quantum computing can be found in [20].

B. Variational quantum circuits

A variational quantum circuit is composed of three steps. The first deals with loading the input data. The second deals with the ways in which circuit parameters are carried, consisting of parameterized quantum gates. The third stage is the reading/measurement stage, so that the result is extracted from the circuit. The parameter layer can be repeated to introduce more variables into the system and more diversity in the circuit's behavior. Figure 2 displays the representation of a VQC.

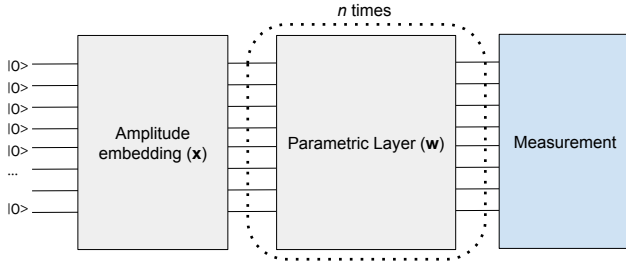


Fig. 2. Variational Quantum Circuit Design. The *Amplitude embedding* box represents the set of operations required to load the input vector $\mathbf{x}$ as probability amplitudes of a given quantum state. The *Parametric Layer* box represents the set of operations that manipulate the parameters $\mathbf{w}$ of the system in order to allow the circuit to perform different mathematical operations. The *measurement* stage offers the final result to whoever is carrying out the experiment.

IV. GENETIC ALGORITHMS

Genetic algorithms (GAs) constitute a category of evolutionary search techniques renowned for their efficacy in navigating high-dimensional search spaces. When applied to real-parameter problems, GAs typically encode each parameter as a bit string, utilizing either standard binary coding or Gray coding methodologies. These bit strings, representing parameters, are concatenated to form a singular bit string, termed a "chromosome," which encapsulates the entire parameter vector. Borrowing from biological parlance, each bit position corresponds to a gene within the chromosome, while each bit value corresponds to an allele. Denoting a parameter vector as $\mathbf{x}$, the corresponding bit string is represented by the uppercase letter $\mathbf{X}$. Consequently, the problem transforms into a combinatorial challenge, with the search space defined by the vertices of a high-dimensional cube [21].

V. PROPOSED APPROACH

In this section, we present the proposed approach to use genetic algorithms to find weights of a variational circuit. In our proposal, we carried out experiments on 6 different variational layer types, applied to 3 different databases. The results found by GA were compared with 2 classic optimizers, Adam [22] and Nesterov Momentum [23]. In the next subsections, we will detail the proposed experimental environment. We also evaluated at what point the algorithms (genetic and classical optimizing) converged.

A. Circuit Encoding and Pre-processing Stage

The feature encoding used for the quantum circuits was accomplished using the Amplitude Embedding technique from the PennyLane library [24]. This method allows for encoding 2^n features into the amplitude vector of n qubits, enabling the circuit to directly leverage the quantum superposition principle. Equation 7 shows the loading of a vector of N values, $\chi_1, \chi_2, \dots, \chi_N$ for a given basis states $|0\rangle, |1\rangle, \dots, |N-1\rangle$.

$$|\psi\rangle = \frac{1}{\sqrt{\sum_b^N \chi_b^2}} \sum_b^N \chi_b |b\rangle \quad (7)$$

Prior to encoding, each dataset's features underwent pre-processing steps, which included padding to ensure consistent dimensionality (as a final value always power of 2) and normalization of features between 0 and 1.

B. Optimizers

To compare the performance of GA with traditional gradient-based methods, we selected two prominent optimizers: Adam and Nesterov Momentum. Both optimizers have demonstrably successful track records in classical machine learning tasks, making them ideal benchmarks for assessing the efficacy of GA in the quantum setting. For each optimizer, we evaluated its performance across a range of learning rates (0.01, 0.05, and 0.1) and different number of iterations (50, 150 and 200). This comprehensive experimental design allowed us to identify the optimal configurations for each optimizer and compare their effectiveness against GA.

C. Genetic Algorithm Setup

To guide our circuit optimization within the quantum setting, we employed a robust genetic algorithm (GA) implemented through the PYGAD library in Python [25]. GA allowed us to evaluate each circuit architecture for each dataset, exploring an extensive parameter space.

Specifically, we executed the GA for three population sizes (50, 150, and 200 individuals) while considering two parent selection methods (tournament and steady-state) and a combination of crossover (single-point and two-points crossover) and mutation operators (random mutation and adaptive mutation), to explore different genetic manipulations.

The goal of the fitness function is to evaluate each candidate solution in terms of its ability to accurately classify the training data using a variational quantum circuit. By assigning a fitness score based on accuracy, the genetic algorithm can iteratively improve the weights and bias, ultimately finding an optimal or near-optimal set of parameters that maximizes the classification performance.

D. Dataset Configuration

For the classification task, we chose three different datasets, each with two classes, from the UCI Machine Learning Repository. The Sonar dataset, the largest one in terms of the number of features, has 60 features and 208 samples. The Diagnosis Breast Cancer dataset includes 31 features and

570 samples. The final dataset, the Banknotes Authentication dataset, is the smallest in terms of the number of features, with 4 features and 1373 samples.

To ensure better performance and comparison across models, we applied normalization and class balancing as preprocessing steps. All datasets were normalized using min-max scaling to ensure feature values contributed equally to the model training process. The Diagnosis Breast Cancer dataset, which presented class imbalance, was balanced using SMOTE (Synthetic Minority Over-sampling Technique) [26], improving the representation of the minority class. To evaluate the generalization capability of each model, we employed a 10-fold cross-validation strategy.

Feature engineering was also employed to enhance the predictive power by selecting the most informative features for each dataset. These steps were essential in preparing the datasets effectively for training and improving the overall classification accuracy of the models.

E. Layers

To evaluate the performance of GA in different scenarios, we investigated six different layer configurations described in [27]. Figures 3, 4, 5, 6, 7, and 8 show what is inside the gate box of the *Parametric Layer* for a given parametric circuit. As one can see, these are circuits with different numbers of operators and with diverse operations.

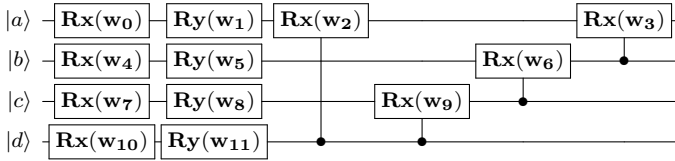


Fig. 3. Layer 1 for *Parametric Layer* VQC design.

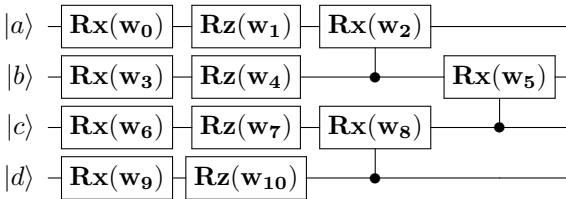


Fig. 4. Layer 2 for *Parametric Layer* VQC design.

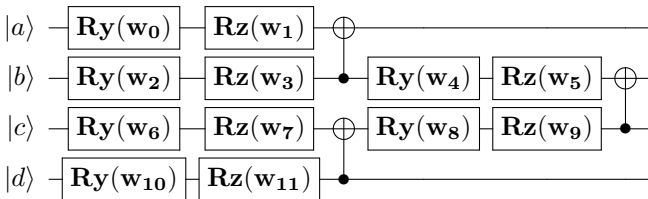


Fig. 7. Layer 5 for *Parametric Layer* VQC design.

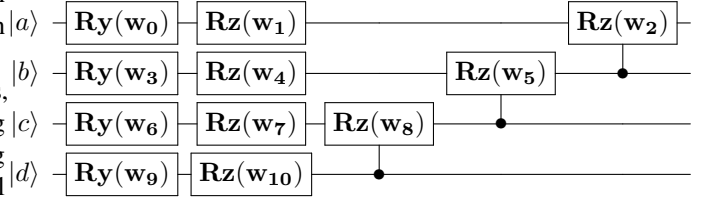


Fig. 8. Layer 6 for *Parametric Layer* VQC design.

VI. RESULTS

Our investigations into the performance of ansatz circuits for sonar data classification revealed a significant dependence on layer size. Layer 5 achieved demonstrably superior generalization performance compared to other tested configurations. The algorithm was executed 200 times, and the average classification accuracies, precision, recall, and F1-score were calculated. For example, when using 6, 15, and 20 layers, we observed a preliminary trend of increasing accuracy, ranging approximately from 72% to 90%. The GA optimizer exhibited significantly faster convergence, achieving peak performance in just 12 training iterations.

For comparison, logistic regression achieves 76% accuracy on this dataset, while a classical multilayer perceptron reaches 92% using K-fold cross-validation. Our VQC-based approach, even when optimized with GA, reaches 88%, demonstrating competitive results using a gradient-free evolutionary optimizer in a quantum setting. All metrics were averaged across 200 independent runs.

The cancer dataset exhibited a significant correlation between classification accuracy and circuit layer size. Layers 5 and 6 achieved the highest average accuracies within this dataset, demonstrating superior performance. The algorithm was executed 200 times, and the average classification accuracies, precision, recall, and F1-score were calculated. Notably, increasing the number of layers from 6 to 15 and then 20 resulted in a progressive rise in model performance, culminating in peak average accuracies of 63%, 72%, and 78%, which reflect the average accuracies across 200 executions, respectively. The best precision, recall, and F1-score achieved were 76%, 77%, and 76.5%, respectively, with the 20-layer configuration. However, compared to logistic regression on this dataset, which achieves an accuracy of 95%, our approach falls short in terms of final accuracy. Nevertheless, our approach achieves faster convergence. The genetic algorithm, for instance, reaches peak accuracy in just 24 training iterations, while both Adam and Nesterov optimizers converge in 100-120 iterations. While the logistic regression model converges faster in terms of iteration count, it is important to note that quantum variational circuits offer advantages in different aspects, such as generalizability in certain domains and flexibility for parameter adaptation.

Intriguingly, the banknote dataset, with its comparatively low feature dimensionality (4 features), did not exhibit a pronounced dependence of classification accuracy on circuit layer size like the other datasets. Layer 5 consistently displayed the highest average performance across all configurations. The average

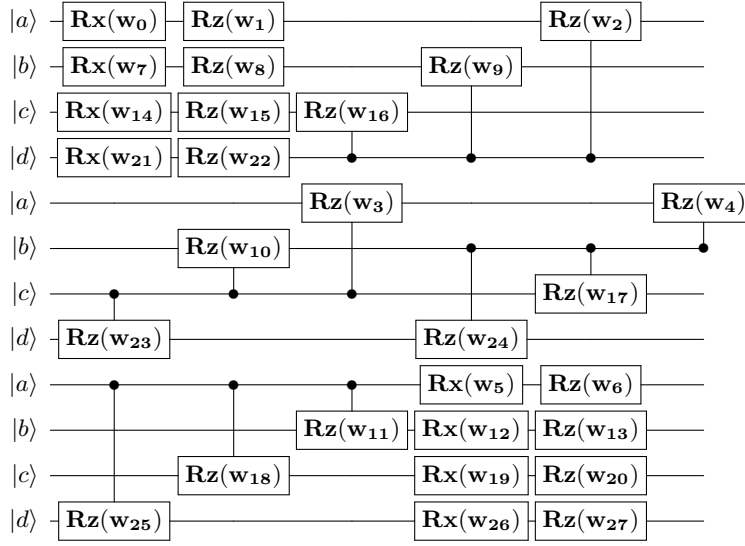


Fig. 5. Layer 3 for *Parametric Layer VQC* design.

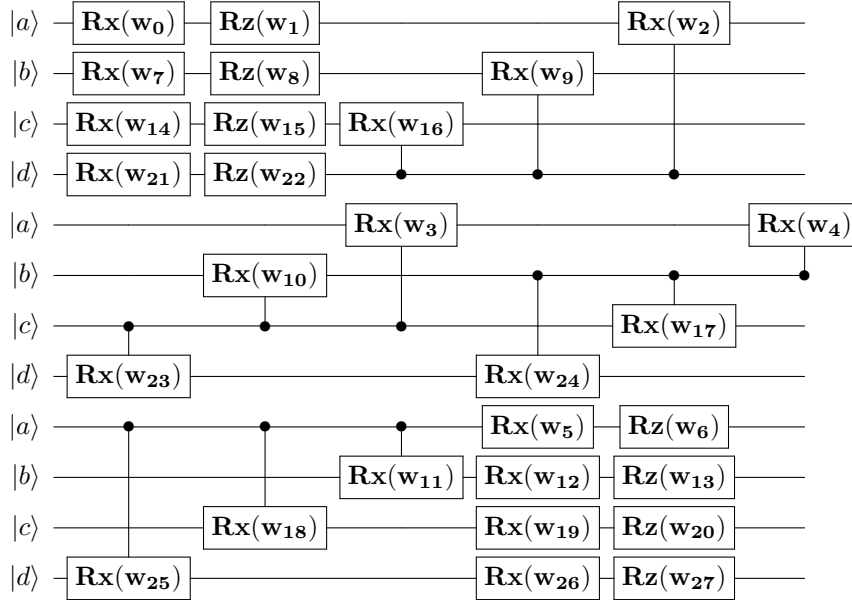


Fig. 6. Layer 4 for *Parametric Layer VQC* design.

classification accuracies, precision, recall, and F1-score were calculated. Increasing the number of layers from 6 to 15 and then 20 resulted in plateauing average accuracy scores of 67%, 73%, and 75%, respectively. The best precision, recall, and F1-score achieved were 74%, 75%, and 74.5%, respectively, with the 20-layer configuration.

To evaluate whether optimizer choice had a statistically significant impact on model performance, we conducted a comprehensive analysis of accuracy, precision, recall, and F1-score across the three datasets (Sonar, Breast Cancer, and Banknotes), using 10-fold cross-validation. Shapiro–Wilk tests confirmed that most metric distributions adhered to the assumption of normality, enabling the use of parametric

methods.

One-way ANOVA revealed that, for the Sonar dataset, there were no statistically significant differences across optimizers in any of the four metrics ($p > 0.1$), although GA presented consistently higher average performance. For the Breast Cancer and Banknotes datasets, however, recall emerged as a metric particularly affected by optimizer choice, with ANOVA indicating significant differences ($p = 0.0213$ and $p = 0.0374$, respectively).

These findings suggest that GA achieved higher average recall in both datasets where statistical significance was observed, which is especially relevant in applications where minimizing false negatives is a priority. Recall is particularly informative in

imbalanced classification problems, as it quantifies the model's ability to correctly identify minority class instances, which are often overlooked in overall accuracy metrics. Furthermore, such a result supports the quality of the GA in working on different problems with equal or superior quality to optimizers based on gradient descent.

The performance of various optimizers compared to GA is presented in Table I for the three datasets, Sonar, Cancer, and Banknotes, respectively. These experiments used a consistent configuration across all datasets: a learning rate of 0.05, a batch size of 16, circuit layer 5 from Figure 7, and 20 layers.

Table II details the exploration of various GA configurations for each dataset, Sonar, Cancer, and Banknotes, respectively. We investigated the impact of different combinations of parent selection methods (tournament and steady-state), crossover methods (single-point and two-point), and mutation methods (random and adaptive). The table presents the total accuracy achieved by each GA configuration with a population size equal to 150.

Table III summarizes the results obtained with the Adam optimizer for different layer configurations across all three datasets, Sonar, Cancer, and Banknotes, respectively. Notably, the Adam optimizer exhibited consistent performance across the evaluated layer sizes within the tested range, suggesting robustness to layer size.

The genetic algorithm exhibited a distinct advantage in optimizing variational circuits over gradient-based methods. This superiority is evident in the experimental results presented in Table I. Notably, the genetic algorithm achieved convergence accuracy comparable to that of traditional optimizers within a significantly reduced number of iterations. The crossover operation was applied with a 80% probability and a 20% probability for mutation operations.

To quantify the improvement in convergence speed, we calculated the ratio between the number of iterations required by the Adam optimizer and the Genetic Algorithm (GA) to reach peak performance on each dataset, showing that GA achieved convergence up to 7.3 times faster for the Sonar dataset (12 vs. 87 iterations), 4.8 times for the Cancer dataset (23 vs. 111), and 3.7 times for the Banknotes dataset (36 vs. 132).

These results align with prior studies emphasizing the strengths of genetic algorithms for quantum optimization tasks. For instance, works such as [13] demonstrated genetic algorithms on limited datasets but lacked broader applicability or comparisons with gradient-based methods. Similarly, while [15] optimized QAOA parameters, our work goes further by generalizing to diverse quantum circuits and datasets, delivering competitive or superior results.

The ability of the genetic algorithm to avoid local minima and mitigate barren plateaus, challenges well-documented in quantum optimization literature, reinforces its robustness. In contrast to approaches like those in [16] and [17], which focus on structural optimization, our method fine-tunes parameters, yielding predictive accuracy that fully leverages the quantum circuit's potential.

Furthermore, our results on the Sonar dataset, where the genetic algorithm outperformed logistic regression, provide empirical evidence of its practical utility, especially for tasks where rapid convergence is essential. This demonstrates that our approach not only complements existing methodologies but also sets a benchmark for efficiency and versatility in training quantum circuits.

VII. CONCLUSION

This work investigated the ability of the Genetic Algorithm to efficiently train weights in a variational classifier, compared to traditional gradient-based optimizers. The results strongly support the initial hypothesis, demonstrating that the genetic algorithm consistently achieved faster convergence across some datasets and configurations. It was possible to conclude that the different genetic algorithm configurations did not significantly change the accuracy, as well as its convergence speed, in the three databases tested. The databases used, namely Sonar, Cancer and Banknote, have very different amounts of features and samples, representing different scenarios for using the algorithm. Compared to two quantum circuit optimizers, Adam and Nesterov, the genetic algorithm showed its convergence speed up 7.3 times (in Sonar Database), 4.8 times (Cancer Database), and 3.7 times (Banknote Database) in the experiments. This significantly improved efficiency suggests the potential for the genetic algorithm to accelerate training times and improve performance in real-world applications of variational circuits.

While a comprehensive exploration of configuration parameters was limited by time constraints, the observed efficiency gains warrant further investigation of the algorithm's performance on specific problem domains and its alignment with theoretical frameworks like population diversity and landscape exploration in evolutionary algorithms. The efficiency advantage of the genetic algorithm opens exciting possibilities for applications in real-world scenarios where fast training times are crucial, such as online learning, resource-constrained settings, and dynamic environments.

Future efforts could be directed toward exploring new evolutionary techniques for optimizing variational circuits and pushing the boundaries of performance. Techniques like Particle Swarm Optimization (PSO) and Differential Evolution (DE) offer intriguing possibilities. PSO's dynamic information sharing mechanism excels in high-dimensional landscapes, while DE's diverse exploration might uncover hidden optima. Moving beyond synthetic data, our aim is to tackle more complex and realistic datasets such as MNIST. This will test the generalizability of our algorithms to diverse classification tasks and push them towards practical applications. A deeper understanding of what gives specific evolutionary techniques their edge is crucial. Exploring the theoretical underpinnings of convergence properties, information exchange mechanisms, and landscape complexity will provide valuable insights for algorithm selection and refinement.

TABLE I
ACCURACY, PRECISION, RECALL, AND F1-SCORE FOR DIFFERENT QUANTUM CIRCUIT OPTIMIZERS FOR DIFFERENT CONFIGURATIONS AND USING CIRCUIT 5 AS QUANTUM VARIATIONAL CIRCUIT APPLIED TO SONAR, CANCER, AND BANKNOTES DATASETS.

Dataset	Optimizer	Iterations	Convergence Iteration	Accuracy (Avg)	Precision (Avg)	Recall (Avg)	F1-score (Avg)
Sonar	Adam	50	50	71%	72%	69%	70%
	Nesterov Momentum	50	50	69%	68%	67%	67.5%
	Adam	200	87	88%	87%	89%	88%
	Nesterov Momentum	200	88	87%	86%	85%	85.5%
	GA	50	12	88%	88%	87%	87.5%
Cancer	Adam	50	50	62%	63%	61%	62%
	Nesterov Momentum	50	50	61%	60%	60%	60%
	Adam	200	111	75%	74%	76%	75%
	Nesterov Momentum	200	111	74%	73%	72%	72.5%
	GA	50	23	76%	75%	77%	76%
Banknotes	Adam	50	50	65%	66%	64%	65%
	Nesterov Momentum	50	50	64%	63%	62%	62.5%
	Adam	200	132	72%	71%	73%	72%
	Nesterov Momentum	200	132	71%	70%	69%	69.5%
	GA	50	36	73%	72%	74%	73%

TABLE II
BEST SOLUTIONS FOR DIFFERENT GA PARAMETERS EXECUTED IN SONAR, CANCER, AND BANKNOTES DATASETS INCLUDING ACCURACY, PRECISION, RECALL, AND F1-SCORE.

Dataset	Parent Selection	Crossover	Mutation	Accuracy (Avg)	Precision (Avg)	Recall (Avg)	F1-score (Avg)
Sonar	tournament	Single-Point	Random	84%	83%	82%	82.5%
	tournament	Single-Point	Adaptive	86%	85%	86%	85.5%
	tournament	Two-Points	Random	85%	84%	85%	84.5%
	tournament	Two-Points	Adaptive	88%	87%	89%	88%
	steady-state	Single-Point	Random	85%	84%	83%	83.5%
	steady-state	Single-Point	Adaptive	87%	86%	85%	85.5%
	steady-state	Two-Points	Random	86%	85%	86%	85.5%
	steady-state	Two-Points	Adaptive	86%	86%	82%	86%
Cancer	tournament	Single-Point	Random	70%	71%	69%	70%
	tournament	Single-Point	Adaptive	73%	72%	74%	73%
	tournament	Two-Points	Random	72%	71%	70%	70.5%
	tournament	Two-Points	Adaptive	76%	75%	77%	76%
	steady-state	Single-Point	Random	72%	73%	71%	72%
	steady-state	Single-Point	Adaptive	74%	73%	75%	74%
	steady-state	Two-Points	Random	74%	73%	72%	72.5%
	steady-state	Two-Points	Adaptive	72%	70%	70%	71%
Banknotes	tournament	Single-Point	Random	69%	68%	67%	67.5%
	tournament	Single-Point	Adaptive	70%	69%	70%	69.5%
	tournament	Two-Points	Random	71%	70%	71%	70.5%
	tournament	Two-Points	Adaptive	73%	72%	74%	73%
	steady-state	Single-Point	Random	70%	69%	68%	68.5%
	steady-state	Single-Point	Adaptive	71%	70%	71%	70.5%
	steady-state	Two-Points	Random	71%	70%	69%	69.5%
	steady-state	Two-Points	Adaptive	73%	70%	68%	71%

TABLE III
ACCURACY, PRECISION, RECALL, AND F1-SCORE FOR VARIOUS LAYER SIZES FOR LAYER 5 CONFIGURATION APPLIED TO THE SONAR, CANCER, AND BANKNOTES DATASETS WITH ADAM OPTIMIZER.

Dataset	Layer Configuration	Convergence Iteration (Avg)	Accuracy (Avg)	Precision (Avg)	Recall (Avg)	F1-score (Avg)
Sonar	6 Layers	105	70%	68%	69%	68.5%
	15 Layers	92	83%	81%	82%	81.5%
	20 Layers	87	88%	87%	89%	88%
Cancer	6 Layers	110	65%	63%	64%	63.5%
	15 Layers	115	73%	71%	72%	71.5%
	20 Layers	111	75%	74%	76%	75%
Banknotes	6 Layers	140	67%	65%	66%	65.5%
	15 Layers	137	73%	72%	73%	72.5%
	20 Layers	132	72%	71%	73%	72%

VIII. ACKNOWLEDGEMENTS

This work was supported by research grants from Fundação de Amparo à Ciência e Tecnologia do Estado de Pernambuco (FACEPE), under Grant APQ-1110-1.03/21; the National Council for Scientific and Technological Development (CNPq); The research cooperation project between Softex (with funding from the Ministry of Science, Technology and Innovation—Law 8.248) and CIn-UFPE..

REFERENCES

- [1] Devitt, Simon J., William J. Munro, and Kae Nemoto. "Quantum error correction for beginners." *Reports on Progress in Physics* 76.7 (2013): 076001.
- [2] Preskill, John. "Quantum computing in the NISQ era and beyond." *Quantum* 2 (2018): 79.
- [3] Leymann, F., and J. Barzen. "The bitter truth about gate-based quantum algorithms in the NISQ era. *Quantum Sci. Technol.* 5 (4), 1–28 (2020)."
- [4] Cerezo, Marco, et al. "Variational quantum algorithms." *Nature Reviews Physics* 3.9 (2021): 625-644.
- [5] Jarrod R McClean et al. "The theory of variational hybrid quantum-classical algorithms". In: *New Journal of Physics* 18.2 (2016), p. 023023
- [6] Kapoor, Ashish, Nathan Wiebe, and Krysta Svore. "Quantum perceptron models." *Advances in neural information processing systems* 29 (2016).
- [7] Sejnowski, Terry and Gorman, R.. *Connectionist Bench (Sonar, Mines vs. Rocks)*. UCI Machine Learning Repository. <https://doi.org/10.24432/C5T01Q>.
- [8] Lohweg, Volker. (2013). *banknote authentication*. UCI Machine Learning Repository. <https://doi.org/10.24432/C55P57>.
- [9] Wolberg, William, Mangasarian, Olvi, Street, Nick, and Street, W.. (1995). *Breast Cancer Wisconsin (Diagnostic)*. UCI Machine Learning Repository. <https://doi.org/10.24432/C5DW2B>.
- [10] Montana, David J., and Lawrence Davis. "Training feedforward neural networks using genetic algorithms." *IJCAI*. Vol. 89. 1989.
- [11] Srinivas, M., and L. M. Patnaik. "Learning neural network weights using genetic algorithms-improving performance by search-space reduction." [Proceedings] 1991 IEEE International Joint Conference on Neural Networks. IEEE, 1991.
- [12] Tsoulos, Ioannis G., et al. "Train RBF networks with a hybrid genetic algorithm." *Evolutionary Intelligence* 16.1 (2023): 375-381.
- [13] Acampora, Giovanni, Angela Chiatto, and Autilia Vitiello. "Training variational quantum circuits through genetic algorithms." 2022 IEEE Congress on Evolutionary Computation (CEC). IEEE, 2022."
- [14] Acampora, Giovanni, Angela Chiatto, and Autilia Vitiello. "Genetic algorithms as classical optimizer for the Quantum Approximate Optimization Algorithm." *Applied Soft Computing* 142 (2023): 110296.
- [15] Giovagnoli, Alessandro, et al. "Qneat: Natural evolution of variational quantum circuit architecture." *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*. 2023.
- [16] Sünkel, Leo, et al. "GA4QCO: genetic algorithm for quantum circuit optimization." *arXiv preprint arXiv:2302.01303* (2023).
- [17] Acampora, Giovanni, Angela Chiatto, and Autilia Vitiello. "A Comparison of Evolutionary Algorithms for Training Variational Quantum Classifiers." 2023 IEEE Congress on Evolutionary Computation (CEC). IEEE, 2023.
- [18] Cao, Yudong, Gian Giacomo Guerreschi, and Alán Aspuru-Guzik. "Quantum neuron: an elementary building block for machine learning on quantum computers." *arXiv preprint arXiv:1711.11240* (2017).
- [19] Acampora, Giovanni, et al. "EVOVAQ: EVOLutionary algorithms-based toolbox for VARIational Quantum circuits." *SoftwareX* 26 (2024): 101756.
- [20] Nielsen, Michael A., and Isaac L. Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.
- [21] Wright, Alden H. "Genetic algorithms for real parameter optimization." *Foundations of genetic algorithms*. Vol. 1. Elsevier, 1991. 205-218.
- [22] Kingma, Diederik P., and Jimmy Ba. "Adam: A method for stochastic optimization." *arXiv preprint arXiv:1412.6980* (2014).
- [23] Xie, Xingyu, et al. "Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models." *arXiv preprint arXiv:2208.06677* (2022).
- [24] Ville Bergholm et al. *PennyLane: Automatic differentiation of hybrid quantum-classical computations*. 2018. *arXiv:1811.04968*
- [25] Gad, Ahmed Fawzy. "Pygad: An intuitive genetic algorithm python library." *Multimedia tools and applications* 83.20 (2024): 58029-58042.
- [26] Chawla, Nitesh V., et al. "SMOTE: synthetic minority over-sampling technique." *Journal of artificial intelligence research* 16 (2002): 321-357.
- [27] Sim, Sukin, Peter D. Johnson, and Alán Aspuru-Guzik. "Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms." *Advanced Quantum Technologies* 2.12 (2019): 1900070.